

HavenWorld

Complete Build Guide

Part 1: Project Foundation & Phase 0 Mac Setup

Developer	Daniel Boone
GitHub Repository	github.com/dboone323/HavenWorld
Game URL (Live)	https://havenworld.pages.dev
Hosting	Cloudflare Pages — free, no domain purchase required
Document Date	September 18, 2026
Document Status	Active — Phase 0 in Progress
Guide Series	4 Parts Total

Internal Development Reference — HavenWorld Alpha Build

Table of Contents

Section 1 — How This Guide Works

- 1.1 The Two-Machine Architecture
- 1.2 Free-Tier-Only Policy
- 1.3 About the Game URL

Section 2 — Free-Tier Tech Stack

2.1 What We Use (All Free) — Full Stack Table

2.2 What We Are NOT Using and Why

2.3 Recommended Monorepo Structure

Section 3 — Phase 0: Mac Tasks

3.1 Local Development Environment

MAC-0-A1 through MAC-0-A10 — Homebrew, nvm, pnpm, Git, SSH, Docker, VS Code

3.2 GitHub Repository Setup

MAC-0-B1 through MAC-0-B11 — Clone, scaffold, configs, branch protection, project board

3.3 Cloud Service Account Setup

MAC-0-C1 through MAC-0-C6 — Cloudflare, Resend, Sentry, UptimeRobot, Discord, Hashnode

3.4 GitHub Secrets and CI/CD Workflows

MAC-0-D1 through MAC-0-D5 — Secrets, deploy-client, deploy-server, PR check, Docker Compose

3.5 Landing Page and Legal

MAC-0-E1 through MAC-0-E4 — Landing page, Terms of Service, Privacy Policy, first push

3.6 Phase 0 Mac Completion Checklist (~40 items)

Continuation Note — Parts 2, 3, and 4 Preview

Section 1

—

How This Guide Works

1.1 The Two-Machine Architecture

All HavenWorld development work is split between exactly two machines. Every step throughout this four-part guide series is tagged with one of the machine labels below. Always check the tag before running any command — some steps are Mac-only, some are Linux-only, and a small number require coordination between both.

Tag	Machine	Role in HavenWorld	What You Do Here
MAC	MacBook Pro (local)	Development, art, client code, repo management	Write all code locally, run local dev server, manage GitHub, create art assets, test in browser. Never runs production workloads.
LINUX	Oracle Cloud ARM Linux VM (remote, always on)	Production server, database, cache, reverse proxy	Runs Node.js + Socket.io game server, PostgreSQL database, Redis cache, and Nginx reverse proxy — 24/7, always on, always free.
BOTH	Coordination steps	Keys, secrets, or configuration shared between machines	Steps that generate a value on one machine and consume it on the other (e.g., SSH deploy keys, GitHub Secrets).

Note — Production vs. Development Separation

The MacBook Pro never runs production workloads. It uses Docker Desktop to run local PostgreSQL and Redis containers for development only. The Oracle VM handles everything production. This separation is intentional and keeps your dev environment clean.

1.2 Free-Tier-Only Policy

This guide uses **only free services** — no credit card charges, no paid plan upgrades, no hidden costs. Every tool, platform, and service listed in this guide is

either free forever or has a permanent free tier that is sufficient to run a 5-25 person alpha test without any cost. The Oracle Cloud Free Tier requires a credit card for identity verification only; nothing is charged and nothing will be charged as long as you stay within the Always Free resource limits (which this guide is designed to do).

If you ever see a paid option offered during setup of any service in this guide, skip it. You do not need it for alpha. The free tiers of every service listed here are adequate for hundreds of concurrent alpha testers, let alone an initial group of 5-25.

Important — Oracle Cloud Credit Card Policy

Oracle Cloud requires a credit card at signup for identity verification purposes. You will **not** be charged during normal Always Free usage. To be safe: after provisioning the VM, go to Oracle Cloud Console → Billing → Budgets and set a budget alert at \$1.00. You will receive an email immediately if any charge is ever about to occur, giving you time to react.

1.3 About the Game URL

HavenWorld does **not** use a purchased domain name. Instead, two free permanent URLs serve the game using native subdomains and a wildcard DNS service. Both are HTTPS, which is a hard requirement — modern browsers refuse WebSocket (WSS) connections over plain HTTP, so every production URL in this stack must be served over HTTPS. Both of the free URLs described below satisfy that requirement at zero cost.

What	URL	How It's Free	HTTPS
Frontend Client	https://havenworld.pages.dev	Cloudflare Pages free subdomain — permanent, global CDN	Included automatically by Cloudflare

What	URL	How It's Free	HTTPS
Game Server API + WSS	https://[oracle-ip].nip.io	nip.io converts any IP to a free domain name — no signup	Let's Encrypt + Certbot on the Oracle VM (free, auto-renews)

nip.io example: If your Oracle Cloud VM public IP address is 129.146.88.42, then your game server API URL is `https://129-146-88-42.nip.io`. The nip.io service simply resolves any IP-based subdomain back to the IP itself — it requires no account, no configuration, and no renewal. Combined with a Let's Encrypt certificate issued by Certbot (covered in Part 2), this gives you a fully valid HTTPS endpoint for free.

Section 2

—

Free-Tier Tech Stack

2.1 What We Use (All Free)

The table below is the complete, authoritative HavenWorld tech stack for alpha. Every service is free within the usage limits shown. No paid plan is required at any point during alpha testing. The table is organized by functional category so you can quickly find the tool responsible for any given concern.

Category	Service / Tool	What It Does	Free Tier Limit	Notes
Game Server Hosting	Oracle Cloud Free Tier (ARM VM)	Runs Node.js game server, PostgreSQL, Redis, Nginx	4 OCPUs + 24 GB RAM + 200 GB storage — forever free	Requires credit card for identity verification only; nothing

Category	Service / Tool	What It Does	Free Tier Limit	Notes
				is charged
Frontend Hosting	Cloudflare Pages	Hosts the Phaser 3 browser client	Unlimited requests and bandwidth	URL: havenworld.pages.dev — no domain purchase needed
Free DNS for IP	nip.io	Converts Oracle VM IP to a domain name	Unlimited, no signup	Format: 129-146-88-42.nip.io
SSL/TLS Certificate	Let's Encrypt + Certbot	Free HTTPS for the game server API	90-day cert; Certbot auto-renews	Applied to the nip.io domain on the Oracle VM
Source Control + CI/CD	GitHub + GitHub Actions	Hosts repo; auto-deploys on git push	Unlimited minutes on public repos	Two workflows: deploy-client → Cloudflare Pages, deploy-server → Oracle VM via SSH
Database	PostgreSQL 16 (self-hosted on Oracle VM)	Stores all game data	No limit — runs on the free VM	No external DB service needed
Cache / Sessions	Redis 7 (self-hosted on Oracle VM)	JWT session store, rate limiting, online status	No limit — runs on the free VM	No Upstash needed
Reverse Proxy + SSL	Nginx (self-hosted on Oracle VM)	Routes HTTPS traffic to Node.js; enables WSS	Free open-source	Handles WebSocket upgrade headers
Process Manager	PM2 (self-hosted on Oracle VM)	Keeps Node.js alive 24/7, restarts on crash	Free open-source	Auto-starts on VM reboot
Transactional Email	Resend	Account verification, alpha invite emails	3,000 emails/month, 100/day	Send from: onboarding@resend.dev (pre-verified, no domain)

Category	Service / Tool	What It Does	Free Tier Limit	Notes
				needed)
Error Monitoring	Sentry	Catches server and client errors in real time	5,000 errors/month	Two projects: havenworld-server and havenworld-client
Uptime Monitoring	UptimeRobot	Alerts you if the server or frontend goes down	50 monitors, 5-minute checks	Sends alerts to email and Discord webhook
Community Hub	Discord	Alpha tester coordination, bug reports, dev log	Free forever	Primary communication channel with testers
Dev Blog	Hashnode	Public development log and game announcements	Free, permanent URL	URL: havenworld.hashnode.dev
IDE	VS Code	Code editor on the MacBook	Free	With specific extensions listed in MAC-0-A9
Local Dev Database	Docker Desktop	Runs PostgreSQL + Redis locally on Mac for development	Free for personal use	Does NOT go to production; Oracle VM has its own DB
Map Editor	Tiled	Creates tilemaps for game rooms	Free / pay-what-you-want	Exports JSON for Phaser 3
Sprite Tool	LibreSprite	Pixel art editor for avatar and furniture sprites	100% free open-source	Fork of old Aseprite
Atlas Packer	TexturePacker (free tier)	Packs sprites into GPU-efficient atlases	Free: up to 2 sprite sheets	Sufficient for Phase 1 alpha
Runtime	Node.js 22 LTS via nvm	JavaScript server runtime	Free open-source	Use nvm to manage versions

Category	Service / Tool	What It Does	Free Tier Limit	Notes
				across environments
Game Engine	Phaser 3	HTML5 2D game engine for the browser client	Free open-source MIT	WebGL + Canvas fallback

2.2 What We Are NOT Using (and Why)

Several popular hosting and infrastructure platforms were considered and deliberately excluded. Every exclusion is based on a concrete technical or cost reason that would negatively impact HavenWorld's alpha experience. This table documents those decisions so future contributors understand the rationale.

Service	Reason Excluded
Railway	Removed free tier; requires \$5/month minimum — violates free-tier-only policy
Heroku	Removed free tier entirely in 2022 — no longer a free option
Render.com	Free tier sleeps after 15 minutes of inactivity — fatal for WebSocket connections , which must remain persistent
Fly.io	Free tier is limited and unpredictable; Oracle Cloud ARM is strictly more powerful (4 OCPUs, 24 GB RAM) and always free
Neon.tech	External DB with cold starts and 0.5 GB storage limit; Oracle VM self-hosted PostgreSQL is faster with no storage cap
Upstash Redis	10,000 commands/day limit is too low for active gameplay; Oracle VM Redis has no command limit
Vercel	Frontend-only platform; Cloudflare

Service	Reason Excluded
	Pages is equivalent in features and is free — no reason to use both
havenworld.me domain	Requires annual purchase; Cloudflare Pages .pages.dev subdomain is free and permanent with identical functionality

2.3 Recommended Monorepo Structure

HavenWorld uses a **pnpm workspace monorepo** — a single Git repository containing all three packages: the game server, the browser client, and a shared TypeScript package. This structure allows the server and client to share types and Socket event constants without duplication. The directory tree below is the target structure you will scaffold in step MAC-0-B2.

```
HavenWorld/ |— apps/ |   |— server/                                # [LINUX] Node.js +
Express + Socket.io game server |   |   |— src/ |   |   |— index.ts
# App entry point |   |   |— routes/                # REST API (auth, users,
rooms, admin) |   |   |— sockets/                # Socket.io event handlers |
|   |   |— services/                # RoomManager, ModerationService |   |
|— middleware/                # requireAuth, requireRole |   |— scripts/
# generate-invites.ts, seed helpers |   |— prisma.ts                #
Prisma client singleton |   |   |— redis.ts                # Redis client
singleton |   |— prisma/ |   |   |— schema.prisma                # Database
schema |   |   |— migrations/                # Auto-generated migration files |
|   |   |— seed.ts                # Seeds rooms, items, default data |   |—
.env.example                # Template (safe to commit – no real secrets) |   |
|— package.json |   |— client/                                # [MAC] Phaser 3 + Vite
browser client |   |   |— src/ |   |   |— main.ts                # Phaser
game config entry point |   |   |— scenes/                # BootScene,
LoginScene, LobbyScene, RoomScene, UIScene |   |   |— entities/
# Avatar.ts, AvatarAnimations.ts |   |   |— services/                #
auth.ts, socket.ts |   |   |— ui/                # AvatarCustomizer.ts,
FriendsList.ts |   |   |— style.css                # All UI styling |   |
|— assets/ |   |   |— sprites/ |   |   |— avatar/                #
Body, eyes, hair, clothing PNGs + atlas.json |   |   |— tilesets/                #
furniture/ # Furniture PNGs + atlas.json |   |   |— maps/                # lobby.json,
grass.png, interior.png |   |   |— index.html                #
town-square.json, park.json, cafe.json |   |   |— .env.development                #
Game shell with all HTML overlay panels |   |   |— .env.production                #
VITE_SERVER_URL=http://localhost:3000 |   |   |— package.json |— packages/ |
VITE_SERVER_URL=https://[ip].nip.io |   |   |— shared/                # [BOTH] Shared TypeScript types and Socket
event constants |   |   |— src/ |   |   |— events.ts                #
SOCKET_EVENTS constants (prevents typos) |   |   |— types.ts                #
PlayerState, AvatarData, RoomData, ChatMessage interfaces |   |   |—
index.ts                # Re-exports everything |   |— package.json
```

```
|— .github/ | |— workflows/ | | |— deploy-client.yml # Pushes
client build to Cloudflare Pages | | |— deploy-server.yml # SSH
deploys server to Oracle VM via appleboy/ssh-action | | |— pr-check.yml
# Lint + type-check + test on all PRs | |— ISSUE_TEMPLATE/ | |—
bug_report.md | |— feature_request.md |— docker-compose.dev.yml
# [MAC] Local PostgreSQL + Redis for development |— ecosystem.config.js
# [LINUX] PM2 configuration for production server |— ROADMAP.md |—
CONTRIBUTING.md |— SECURITY.md |— package.json # pnpm
workspace root
```

Section 3

Phase 0: Mac Tasks

[MAC]

Complete ALL Phase 0 Mac tasks before writing any game code. These steps establish your development environment, repository structure, cloud service accounts, CI/CD pipelines, and legal pages. Estimated time: **3-5 days** working through sections systematically. Do not skip steps — each step is a prerequisite for steps that follow it.

Tip — Recommended Order

Work through the sections in order: 3.1 (dev tools) → 3.2 (repo structure) → 3.3 (cloud accounts) → 3.4 (CI/CD) → 3.5 (landing page). Section 3.3 requires your GitHub repo to exist (from 3.2), and Section 3.4 requires credentials from both 3.2 and 3.3.

3.1 Local Development Environment MAC

MAC-0-A1 — Install Homebrew

Open Terminal (Cmd+Space → type "Terminal" → press Enter). Run the following command — it is the official Homebrew installer and must be run exactly as shown:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

The installer will prompt for your macOS administrator password. After installation completes, **read the printed output carefully** — on Apple Silicon Macs (M1/M2/M3/M4), Homebrew installs to `/opt/homebrew` instead of `/usr/local`, and the installer prints two commands you must run to add Homebrew to your PATH. Run those commands before continuing.

Verify: `brew --version` → should print Homebrew 4.x.x

MAC-0-A2 — Install nvm and Node.js 22 LTS

nvm (Node Version Manager) allows you to install and switch between multiple Node.js versions. Install nvm first:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

Close and reopen Terminal (or run `source ~/.zshrc`). Then install and activate Node.js 22 LTS:

```
nvm install 22 nvm use 22 nvm alias default 22 node --version # should print  
v22.x.x npm --version # should print 10.x.x
```

Using `nvm alias default 22` ensures Node.js 22 is used automatically every time you open a new terminal session.

MAC-0-A3 — Install pnpm

```
npm install -g pnpm pnpm --version # should print 9.x.x
```

Why pnpm instead of npm? pnpm handles monorepos (multiple packages in one repository) efficiently through workspace linking. When you run `pnpm install` from the repo root, it installs dependencies for all three packages (server, client, shared) simultaneously and links them so they can reference each other as if published to npm. npm workspaces work but are significantly slower and less reliable for this use case.

MAC-0-A4 — Install and Configure Git

Git comes pre-installed on macOS. Check with: `git --version`

If Git is not installed (uncommon on modern macOS), run: `xcode-select --install` — this installs Xcode Command Line Tools which includes Git.

Configure your identity (this information appears in every commit you make):

```
git config --global user.name "Daniel Boone" git config --global user.email  
"dboone9@msn.com" git config --global core.autocrlf input git config --global  
pull.rebase true git config --global init.defaultBranch main
```

`core.autocrlf input` prevents Windows-style line endings from being introduced if you ever edit files on a Windows machine. `pull.rebase true` keeps the commit history linear by rebasing instead of merge-committing on pull.

MAC-0-A5 — Generate SSH Key and Add to GitHub

This key identifies your MacBook to GitHub for git operations (clone, push, pull).

```
ssh-keygen -t ed25519 -C "dboone9@msn.com" # press Enter for all prompts
eval "$(ssh-agent -s)" ssh-add ~/.ssh/id_ed25519 pbcopy < ~/.ssh/id_ed25519.pub
# copies public key to clipboard
```

In GitHub: **Settings** → **SSH and GPG Keys** → **New SSH Key** → Title:
"MacBook Pro" → paste → Save.

Verify: `ssh -T git@github.com` → should print: *"Hi dboone323! You've successfully authenticated..."*

MAC-0-A6 — Generate a Separate Deploy Key for GitHub Actions → Oracle VM SSH

This is a **separate** key from your personal GitHub SSH key. It is used exclusively by GitHub Actions to SSH into the Oracle VM and run deployment commands automatically on every push to `main`. Keep the private half secret — it goes into a GitHub Secret in step MAC-0-D1.

```
ssh-keygen -t ed25519 -f ~/.ssh/havenworld_deploy -C "github-actions-deploy" -N ""
```

This creates two files:

- `~/.ssh/havenworld_deploy` — **private key** — will go into GitHub Secret `ORACLE_SSH_KEY`
- `~/.ssh/havenworld_deploy.pub` — **public key** — will go onto the Oracle VM's `~/.ssh/authorized_keys` during Phase 0 Linux setup (Part 2)

Copy the private key to your clipboard for safekeeping now:

```
cat ~/.ssh/havenworld_deploy | pbcopy
```

Warning — Keep This Key Private

Never commit `~/.ssh/havenworld_deploy` to the repository. Never paste it into a chat, email, or document. It goes only into a GitHub Actions Secret (encrypted at rest by GitHub). Anyone with this key can SSH into your Oracle VM as the `ubuntu` user.

MAC-0-A7 — Install Docker Desktop

Download from: <https://www.docker.com/products/docker-desktop/> — choose **Apple Silicon** or **Intel** depending on your Mac model (check: Apple menu → About This Mac → Chip shows "Apple M..." for Apple Silicon).

Install and open Docker Desktop. Wait for the whale icon to appear in the menu bar and show "Docker Desktop is running." This may take 30–60 seconds on first launch.

```
docker --version          # e.g., Docker version 27.x.x
docker compose version    #
e.g., Docker Compose version v2.x.x
```

Docker runs PostgreSQL and Redis containers locally on the Mac for development. **These local containers are for development only** — they never go to production. The Oracle VM has its own PostgreSQL and Redis installed natively.

MAC-0-A8 — Install VS Code

Download from: <https://code.visualstudio.com/> → Download for macOS. Open the downloaded `.zip`, drag **Visual Studio Code.app** to your Applications folder.

Open VS Code. Then enable the code terminal command:

Open Command Palette: **Cmd+Shift+P** → type "Shell Command" → select **"Install 'code' command in PATH"**

Verify: In Terminal, run `code --version` → should print the VS Code version number.

MAC-0-A9 — Install VS Code Extensions

Open the Extensions panel in VS Code (**Cmd+Shift+X**). Search for each extension by its ID (shown in parentheses) and install it. Using the extension ID ensures you install the correct extension, not a similarly named imitation.

#

Extension Name

Extension ID

Purpose

1

ESLint

dbaeumer.vscode-eslint

JavaScript/TypeScript linting — catches errors before runtime

2

Prettier

esbenp.prettier-vscode

Automatic code formatting on save — enforces consistent style

3

Prisma

Prisma.prisma

schema.prisma syntax highlighting and autocomplete

4

GitLens

eamodio.gitlens

Powerful Git history and blame annotations inline in the editor

5

Thunder Client

rangav.vscode-thunder-client

API testing (REST client) without needing Postman

6

Docker

ms-azuretools.vscode-docker

Manage Docker containers and images directly from VS Code

7

DotENV

mikestead.dotenv

.env file syntax highlighting — makes secrets files easier to read

8

Error Lens

usernamehw.errorlens

Shows ESLint/TypeScript errors inline in the editor on the offending line

MAC-0-A10 — Configure VS Code settings.json

Open Command Palette (**Cmd+Shift+P**) → type "Open User Settings (JSON)" → select it. Replace or merge the file contents with:

```
{  "editor.formatOnSave": true,  "editor.defaultFormatter": "esbenp.prettier-vscode",  "editor.tabSize": 2,  "editor.rulers": [100],  "typescript.preferences.importModuleSpecifier": "relative",  "eslint.validate":  ["javascript", "typescript"] }
```

`formatOnSave: true` runs Prettier automatically every time you save a file — you never have to manually format code. `rulers: [100]` shows a vertical line at 100 characters to match the project's line-length limit.

`importModuleSpecifier: relative` ensures TypeScript auto-imports use relative paths consistent with the monorepo structure.

3.2 GitHub Repository Setup MAC

MAC-0-B1 — Clone the Repository

```
cd ~/Projects # create this folder first if it doesn't exist: mkdir ~/Projects git clone
git@github.com:dboone323/HavenWorld.git cd HavenWorld code . # opens
the repo in VS Code
```

Note — Repository Must Exist on GitHub First

If the repository does not yet exist on GitHub, create it at github.com/new before running these commands. Initialize it with a README and a .gitignore (Node template). Make it Public so GitHub Actions has unlimited CI/CD minutes.

MAC-0-B2 — Scaffold the Monorepo Folder Structure

Run all of the following commands from the `HavenWorld/` repo root in Terminal. These create the complete directory tree shown in Section 2.3.

```
mkdir -p apps/server/src/{routes,sockets,services,middleware,scripts} mkdir -p
apps/server/prisma mkdir -p apps/client/src/{scenes,entities,services,ui} mkdir -p
apps/client/src/assets/{sprites/{avatar/{body,eyes,hair,clothing},furniture,tilesets},m
aps} mkdir -p packages/shared/src mkdir -p
.github/{workflows,ISSUE_TEMPLATE}
```

Verify the structure was created: `find . -type d | grep -v node_modules | grep -v .git`

MAC-0-B3 — Create Root package.json (pnpm Workspaces)

Create `HavenWorld/package.json` — this is the monorepo root configuration that tells pnpm to treat `apps/*` and `packages/*` as linked workspace packages:

```
{ "name": "havenworld", "private": true, "engines": { "node": ">=22.0.0", "pnpm":
">=9.0.0" }, "packageManager": "pnpm@9.0.0", "workspaces": ["apps/*",
"packages/*"], "scripts": { "dev": "pnpm --parallel --filter './apps/*' dev",
"build": "pnpm --filter './apps/*' build", "test": "pnpm --filter './apps/*' test", "lint":
```

```
"pnpm --filter './apps/*' lint" } }
```

Run `pnpm install` from the repo root to initialize all workspaces.

MAC-0-B4 — Update `.gitignore`

Open the existing `.gitignore` and ensure all of the following lines are present (add any that are missing):

```
node_modules/ dist/ .env .env.local .env.*.local *.log .DS_Store coverage/ .pnpm-  
store/
```

The `.env` entries are critical — they prevent real secrets from ever being committed to the repository. The `.env.example` files (which contain only placeholder values, no real secrets) are intentionally NOT in `.gitignore` so they can be committed as templates.

MAC-0-B5 — Create `README.md`

Create `HavenWorld/README.md` with the following sections and content:

```
# HavenWorld A browser-based social virtual world — open source, community-  
driven. **Game URL:** https://havenworld.pages.dev ## Tech Stack Node.js 22 |  
Socket.io 4 | Phaser 3 | PostgreSQL 16 | Redis 7 | Vite 6 | Cloudflare Pages ##  
Local Setup **Prerequisites:** Node.js 22+, pnpm 9+, Docker Desktop 1. git  
clone git@github.com:dboone323/HavenWorld.git && cd HavenWorld 2. pnpm  
install 3. docker compose -f docker-compose.dev.yml up -d 4. cp  
apps/server/.env.example apps/server/.env # then fill in values 5. cd apps/server  
&& npx prisma migrate dev 6. pnpm dev # from repo root — starts both server  
and client ## Community Discord: [invite link — add after creating Discord server]
```

License GPL-3.0

MAC-0-B6 — Create ROADMAP.md

Create HavenWorld/ROADMAP.md using GitHub checkbox format so items render as interactive checkboxes on GitHub:

```
# HavenWorld Roadmap ## Phase 0 — Foundation (Current) - [ ] Local dev
environment (Mac) - [ ] Oracle Cloud VM provisioned (Linux) - [ ] PostgreSQL +
Redis on Oracle VM - [ ] Nginx + SSL on Oracle VM - [ ] CI/CD pipelines
configured - [ ] Landing page at havenworld.pages.dev ## Phase 1 — Core Loop
Alpha - [ ] User registration with invite codes - [ ] Email verification - [ ] JWT
authentication - [ ] Avatar creation and customization (9 layers) - [ ] 4 public rooms
with Phaser tilemaps - [ ] Real-time player movement sync - [ ] Real-time chat with
profanity filter - [ ] Personal room with furniture placement - [ ] Friend system with
online status - [ ] 5–25 testers playing successfully ## Phase 2 — Economy and
Mini-games (Future) - [ ] Virtual currency system - [ ] Player-to-player trading - [ ]
Fishing mini-game - [ ] First seasonal event - [ ] Badge and achievement system
```

MAC-0-B7 — Create CONTRIBUTING.md

```
# Contributing to HavenWorld ## Getting Started 1. Fork the repo on GitHub 2.
Clone your fork: git clone git@github.com:YOUR_USERNAME/HavenWorld.git 3.
Set up the local dev environment (see README.md) 4. Create a feature branch
from develop: git checkout -b feature/your-feature-name ## Branching Convention
- main — production only (protected; PRs required) - develop — integration
branch (all features merge here first) - feature/* — new features - fix/* — bug
```

fixes - hotfix/* — emergency production fixes ## Pull Request Requirements - PR must target the develop branch (not main) - PR title format: feat: add avatar layering | fix: chat rate limit | docs: update readme - All CI checks must pass (lint, type-check, tests, build) - Include screenshots for any UI changes - Unit tests required for new server-side logic ## Code Style - ESLint + Prettier enforced automatically on save - Tab size: 2 spaces - Line length: 100 characters maximum

MAC-0-B8 — Create SECURITY.md

Security Policy ## Reporting Vulnerabilities Do NOT open a public GitHub issue for security vulnerabilities. Email: dboone9@msn.com — Response within 72 hours. ## Supported Versions Only the latest commit on main is actively supported during alpha. ## License HavenWorld is GPL-3.0 licensed. All contributions must be compatible with GPL-3.0.

MAC-0-B9 — Create GitHub Issue Templates and PR Template

File: .github/ISSUE_TEMPLATE/bug_report.md

--- name: Bug Report about: Report a bug in HavenWorld --- **Describe the bug:** (clear description of what went wrong) **Steps to reproduce:** 1. Go to... 2. Click... 3. See error **Expected behavior:** (what should have happened) **Screenshots:** (if applicable) **Environment:** Browser name/version, OS, screen resolution

File: .github/ISSUE_TEMPLATE/feature_request.md

--- name: Feature Request about: Suggest a new HavenWorld feature ---

****Problem this solves:**** ****Proposed solution:**** ****Alternatives considered:****

File: .github/PULL_REQUEST_TEMPLATE.md

Summary of Changes (What does this PR do and why?) ## Type of Change - [] Bug fix - [] New feature - [] Refactor - [] Documentation ## Testing - [] pnpm test passes locally - [] Manually tested in Chrome - [] Manually tested in Firefox ## Screenshots (for UI changes)

MAC-0-B10 — Configure Branch Protection on GitHub and Create develop Branch

Go to: **github.com/dboone323/HavenWorld** → **Settings** → **Branches** → **Add branch protection rule**

- Branch name pattern: `main`
- Check: **Require a pull request before merging**
- Check: **Require status checks to pass before merging** (add `pr-check` after the first workflow run)
- Check: **Do not allow bypassing the above settings**
- Click **Save changes**

Create the `develop` branch:

```
git checkout -b develop git push origin develop
```

All future feature work branches off `develop`. Only tested, reviewed code gets promoted to `main` via pull request.

MAC-0-B11 — Create GitHub Project Board

Go to: **github.com/dboone323** → **Projects** → **New project** → **Board template**

- Name: **HavenWorld Alpha Development**
- Columns: **Backlog** | **In Progress** | **In Review** | **Done**

Add labels in the repo (Settings → Labels → Create label for each):

Label Name

Color Hex

Purpose

bug

#d73a4a

Something is broken

enhancement

#0075ca

New feature or improvement

alpha-blocker

#e4e669

Must be resolved before alpha launch

good first issue

#7057ff

Good for new contributors

documentation

#0075ca

3.3 Cloud Service Account Setup MAC

MAC-0-C1 — Create Cloudflare Account and Pages Project

1. Go to: <https://dash.cloudflare.com/sign-up> — no credit card needed. Sign up with email. Verify email.
2. In dashboard: **Pages** → **Create a project** → **Connect to Git** → **GitHub**
3. Authorize Cloudflare access. Select: **dboone323/HavenWorld**
4. Project name: `havenworld` — this creates: `https://havenworld.pages.dev`
5. Build settings: Framework preset: **None** Build command: `pnpm --filter client build` Root directory: (leave blank) Build output directory: `apps/client/dist`
6. Environment Variables tab → Add: `NODE_VERSION = 22`
7. Click **Save and Deploy**

Get credentials for GitHub Actions:

- **API Token:** Cloudflare Dashboard → My Profile → API Tokens → Create Token → use "Edit Cloudflare Pages" template → name it `github-actions` → copy token
- **Account ID:** Cloudflare Dashboard right sidebar → copy Account ID

Save both values — they become GitHub Secrets `CF_API_TOKEN` and `CF_ACCOUNT_ID`.

MAC-0-C2 — Create Resend Account (Free Email Sending)

1. Go to: <https://resend.com> → Sign up free
2. **API Keys → Create API Key** → name: `havenworld-production` → copy key (starts with `re_`)
3. Since you don't have a custom domain, send FROM:
`onboarding@resend.dev`

`onboarding@resend.dev` is Resend's pre-verified shared sender domain — it works immediately with no domain setup required. This is appropriate for an alpha group of 5–25 people. Free tier allows 3,000 emails/month and 100 emails/day — far more than needed for alpha.

Save the API key — it becomes GitHub Secret `RESEND_API_KEY`.

MAC-0-C3 — Create Sentry Account (Free Error Monitoring)

1. Go to: <https://sentry.io> → Sign up free with GitHub
2. Create Organization: **HavenWorld**
3. Create Project 1: Platform = **Node.js** | Name = `havenworld-server` → copy the DSN URL
4. Create Project 2: Platform = **Browser (JavaScript)** | Name = `havenworld-client` → copy the DSN URL
5. Settings → Auth Tokens → Create New Token → name: `github-source-maps` → copy token

Save all three values — they become GitHub Secrets: `SENTRY_DSN_SERVER`, `SENTRY_DSN_CLIENT`, and `SENTRY_AUTH_TOKEN`.

MAC-0-C4 — Create UptimeRobot Account (Free Uptime Monitoring)

1. Go to: <https://uptimerobot.com> → Sign up free
2. After the Oracle VM is live (Part 2 — Phase 0 Linux steps), add two monitors: HTTPS monitor: <https://havenworld.pages.dev> HTTPS monitor: [https://\[your-oracle-ip\].nip.io/health](https://[your-oracle-ip].nip.io/health)
3. Alert contacts: add your email address
4. For Discord alerts: Discord server → #alerts channel → Integrations → Create Webhook → copy URL → add as UptimeRobot alert contact

Free tier provides 50 monitors with 5-minute check intervals — far more than you need for alpha.

MAC-0-C5 — Create Discord Server

In Discord: + **(Add a Server)** → **Create My Own** → **For me and my friends** → **Name: HavenWorld**

Create channels in this order, grouped under the named categories:

Category

Channel

Access

Purpose

INFORMATION

#announcements

@Developer posts; everyone reads

Official game announcements

INFORMATION

#roadmap

Read-only

Pin ROADMAP.md contents here

INFORMATION

#rules

Read-only

Community conduct rules

ALPHA TESTING

#tester-chat

@Alpha Tester role only

Tester discussion

ALPHA TESTING

#bug-reports

@Alpha Tester role only

Structured bug reporting

ALPHA TESTING

#feedback

@Alpha Tester role only

General feedback

ALPHA TESTING

#suggestions

@Alpha Tester role only

Feature suggestions

COMMUNITY

#general

Open to all

General chat

COMMUNITY

#dev-log

Open to all

Daniel posts weekly updates

COMMUNITY

#off-topic

Open to all

Off-topic conversation

STAFF

#moderator-chat

@Developer + @Moderator only

Private staff coordination

STAFF

#report-review

@Developer + @Moderator only

Review submitted reports

STAFF

#alerts

@Developer + @Moderator only

UptimeRobot webhook target

Create roles (Server Settings → Roles → + icon):

Role Name

Color

Key Permissions

@Developer

Gold

Administrator

@Moderator

Blue

Manage Messages, Kick Members

@Alpha Tester

Green

Read ALPHA TESTING channels

@Contributor

Purple

Default (decoration only)

Assign yourself @Developer. Create a permanent invite link (Server Settings → Invites) for general #general access. Create a separate invite that auto-grants @Alpha Tester role for alpha testers.

MAC-0-C6 — Set Up Hashnode Dev Blog (Free)

1. Go to: <https://hashnode.com> → Sign up free
2. Create a new blog. Your URL: **havenworld.hashnode.dev** (free permanent URL)
3. Write and publish the first post: *"Introducing HavenWorld — what we're building and why"*

The first post should cover:

- What HavenWorld is: a browser-based open-source social virtual world
- Inspiration: YoWorld, Club Penguin, Habbo Hotel, MegaPlanet
- Tech stack overview
- Call to action: join Discord and star the GitHub repo

Pin the blog URL in Discord #dev-log and add it to README.md after publishing.

3.4 GitHub Secrets and CI/CD Workflows MAC

MAC-0-D1 — Add All GitHub Secrets

Go to: **github.com/dboone323/HavenWorld** → **Settings** → **Secrets and Variables** → **Actions** → **New repository secret** for each item below:

Secret Name

Value Source

CF_API_TOKEN

Cloudflare Pages API token (from MAC-0-C1)

CF_ACCOUNT_ID

Cloudflare Account ID (from MAC-0-C1)

RESEND_API_KEY

Resend API key (from MAC-0-C2)

SENTRY_DSN_SERVER

Sentry server project DSN (from MAC-0-C3)

SENTRY_DSN_CLIENT

Sentry client project DSN (from MAC-0-C3)

SENTRY_AUTH_TOKEN

Sentry auth token for source maps (from MAC-0-C3)

ORACLE_HOST

Oracle VM public IP address (from Part 2 — Phase 0 Linux setup)

ORACLE_SSH_KEY

Contents of

~/ssh/havenworld_deploy

— run:

cat ~/ssh/havenworld_deploy

JWT_ACCESS_SECRET

Run in Terminal:

openssl rand -hex 64

— copy the output

JWT_REFRESH_SECRET

Run again:

openssl rand -hex 64

— must be different from JWT_ACCESS_SECRET

DATABASE_URL

postgresql://havenworld:DB_PASSWORD@127.0.0.1:5432/havenworld_prod

REDIS_URL

redis://:REDIS_PASSWORD@127.0.0.1:6379

Note — 127.0.0.1 in DATABASE_URL and REDIS_URL

DATABASE_URL and REDIS_URL use 127.0.0.1 (localhost) because the GitHub Actions SSH deploy script runs commands *directly on the Oracle VM* via SSH. From the Oracle VM's perspective, its own PostgreSQL and Redis are on localhost. There is no external database connection.

MAC-0-D2 — Create Client Deploy Workflow

Create file `.github/workflows/deploy-client.yml`. This workflow runs automatically whenever you push changes in `apps/client/` or `packages/shared/` to `main`, and deploys the built client to Cloudflare Pages.

```
name: Deploy Client to Cloudflare Pages on: push: branches: [main] paths:
['apps/client/**', 'packages/shared/**'] jobs: deploy: runs-on: ubuntu-latest
steps: - uses: actions/checkout@v4 - uses: pnpm/action-setup@v4
```

```

with: { version: 9 }    - uses: actions/setup-node@v4    with: { node-version:
22, cache: pnpm }      - run: pnpm install --frozen-lockfile    - run: pnpm --filter
client build    - uses: cloudflare/pages-action@v1    with:    apiToken: $
{{ secrets.CF_API_TOKEN }}    accountId: ${{ secrets.CF_ACCOUNT_ID }}
projectName: havenworld    directory: apps/client/dist    gitHubToken: $
{{ secrets.GITHUB_TOKEN }}

```

MAC-0-D3 — Create Server Deploy Workflow

Create file `.github/workflows/deploy-server.yml`. This workflow runs tests first, then if tests pass, SSHs into the Oracle VM, pulls the latest code, runs database migrations, rebuilds, and restarts PM2.

```

name: Deploy Server to Oracle VM on: push:    branches: [main]    paths:
['apps/server/**', 'packages/shared/**'] jobs: test:    runs-on: ubuntu-latest
steps:    - uses: actions/checkout@v4    - uses: pnpm/action-setup@v4
with: { version: 9 }    - uses: actions/setup-node@v4    with: { node-version:
22, cache: pnpm }    - run: pnpm install --frozen-lockfile    - run: pnpm --filter
server lint    - run: pnpm --filter server test    deploy:    needs: test    runs-on:
ubuntu-latest    steps:    - uses: actions/checkout@v4    - name: SSH Deploy
to Oracle VM    uses: appleboy/ssh-action@v1.0.3    with:    host: $
{{ secrets.ORACLE_HOST }}    username: ubuntu    key: $
{{ secrets.ORACLE_SSH_KEY }}    script: |    cd /opt/havenworld
git pull origin main    pnpm install --frozen-lockfile    cd apps/server
npm prisma migrate deploy    pnpm build    pm2 restart havenworld-
server || pm2 start dist/index.js --name havenworld-server    pm2 save

```

The `needs: test` directive ensures the deploy job only runs if the test job passes. If lint or tests fail, no deployment occurs — protecting production from

broken code.

MAC-0-D4 — Create PR Check Workflow

Create file `.github/workflows/pr-check.yml`. This runs on every pull request targeting `develop` or `main`. All checks must pass before the PR can be merged (enforced by branch protection configured in MAC-0-B10).

```
name: PR Checks on: pull_request: branches: [main, develop] jobs: check:
runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses:
pnpm/action-setup@v4 with: { version: 9 } - uses: actions/setup-node@v4
with: { node-version: 22, cache: pnpm } - run: pnpm install --frozen-lockfile -
run: pnpm --filter server lint - run: pnpm --filter client lint - run: pnpm --filter
server test - run: pnpm --filter server build - run: pnpm --filter client build
```

MAC-0-D5 — Create Docker Compose for Local Development

Create `docker-compose.dev.yml` at the repo root. This spins up local PostgreSQL and Redis containers on your Mac for development. **This is for Mac development only — the Oracle VM has its own native PostgreSQL and Redis.**

```
version: '3.9' services: postgres: image: postgres:16-alpine environment:
POSTGRES_DB: havenworld_dev POSTGRES_USER: postgres
POSTGRES_PASSWORD: devpassword123 ports: - '5432:5432'
volumes: - postgres_dev_data:/var/lib/postgresql/data redis: image:
redis:7-alpine ports: - '6379:6379' command: redis-server --appendonly
yes volumes: - redis_dev_data:/data volumes: postgres_dev_data:
```

redis_dev_data:

Command

What It Does

```
docker compose -f docker-compose.dev.yml up -d
```

Starts PostgreSQL and Redis containers in the background

```
docker compose -f docker-compose.dev.yml down
```

Stops containers (data is preserved in volumes)

```
docker compose -f docker-compose.dev.yml down -v
```

Stops containers AND deletes all data (full reset)

3.5 Landing Page and Legal MAC

MAC-0-E1 — Create the Pre-Launch Landing Page

Before the game client is ready, `havenworld.pages.dev` should display a professional coming-soon page so that anyone who discovers the URL (via GitHub, Discord, Hashnode, or word of mouth) sees a real, polished presence rather than a Cloudflare error page.

Create `apps/client/index.html` as a polished static page with:

- HavenWorld logo/title in large teal text
- Tagline: *"A new kind of social world — coming soon"*
- 2-sentence description of what HavenWorld is
- **Discord button:** "Join our community" → Discord server invite URL
- **GitHub button:** "Star us on GitHub" →

<https://github.com/dboone323/HavenWorld>

- "Alpha Coming Soon" badge
- Footer links: Terms of Service | Privacy Policy

Styling target: dark background (#0d0d1a), teal accent (#4ecdc4), professional game aesthetic. No external dependencies — pure HTML + inline CSS only, so Cloudflare Pages deploys it with zero build step.

MAC-0-E2 — Create Terms of Service

Create `apps/client/terms.html`. Terms of Service must exist before any user creates an account. Minimum required clauses for an alpha:

- **Alpha Software Notice:** Pre-release software; bugs expected; game data may be wiped between updates without notice
- **Age Requirement:** Must be 13 or older to create an account (COPPA compliance)
- **Acceptable Use:** No harassment, hate speech, spam, cheating, or impersonation
- **Account Actions:** HavenWorld may suspend or ban accounts that violate these Terms at any time
- **Data Collection:** Username, email, and gameplay data collected for account management and game functionality
- **No Warranty:** Software provided as-is during alpha; no uptime guarantees are made
- **Contact:** dboone9@msn.com

MAC-0-E3 — Create Privacy Policy

Create `apps/client/privacy.html`. Required before collecting any user data (even just email). Minimum required sections:

- **What Is Collected:** Email address, username, IP address (server logs), in-game activity
- **How It Is Used:** Account creation and security, game functionality, error monitoring via Sentry
- **Third-Party Services:** Sentry (crash reporting), Resend (email delivery) — link each service's own privacy policy
- **Data Retention:** Data is kept until account deletion is requested
- **Your Rights:** Email `dboone9@msn.com` to request account and data deletion
- **No Sale of Data:** No data is sold to third parties under any circumstances

MAC-0-E4 — Commit and Push the Foundation

This is the first real commit — everything you've built in Phase 0 Mac goes to GitHub in one clean commit, triggering the first Cloudflare Pages deployment.

`git add . git commit -m "chore: Phase 0 foundation — monorepo structure, workflows, landing page, legal pages" git push origin develop`

1. Go to `github.com/dboone323/HavenWorld` → **Pull requests** → **New pull request**
2. Base: `main` ← Compare: `develop`
3. Title: *chore: Phase 0 Mac foundation*
4. The PR check workflow will run. The first PR won't yet have the `pr-check` status check added to branch protection — that's expected. Proceed to

merge.

5. Merge the PR. This triggers `deploy-client.yml`.

Result: The landing page will be live at `https://havenworld.pages.dev` within approximately 60 seconds after the merge.

Tip — After First Deploy

Go back to Settings → Branches → branch protection rule for `main` and add `pr-check` as a required status check. It now appears in the dropdown because the first workflow run has completed and GitHub knows the job name.

3.6 Phase 0 Mac Completion Checklist

Use this checklist to track your progress through all Phase 0 Mac tasks. Every item must be checked before beginning Part 2 (Phase 0 Linux — Oracle Cloud VM setup). Do not move to Part 2 with any items incomplete.

Local Development Environment (MAC-0-A)

- [] **MAC-0-A1** — Homebrew installed; `brew --version` prints 4.x.x; PATH configured for Apple Silicon if applicable
- [] **MAC-0-A2** — nvm installed; Node.js 22 LTS installed; `node --version` prints v22.x.x; default alias set
- [] **MAC-0-A3** — pnpm installed globally; `pnpm --version` prints 9.x.x
- [] **MAC-0-A4** — Git configured with name "Daniel Boone", email `dboone9@msn.com`, `autocrlf`, `rebase`, and default branch settings
- [] **MAC-0-A5** — Personal SSH key (ed25519) generated and added to GitHub; `ssh -T git@github.com` authenticates successfully

- [] **MAC-0-A6** — Deploy key (`havenworld_deploy`) generated; private key saved for GitHub Secret; public key ready for Oracle VM
- [] **MAC-0-A7** — Docker Desktop installed and running; whale icon visible in menu bar; `docker --version` and `docker compose version` both print versions
- [] **MAC-0-A8** — VS Code installed; code command available in Terminal; `code --version` prints version
- [] **MAC-0-A9** — All 8 VS Code extensions installed: ESLint, Prettier, Prisma, GitLens, Thunder Client, Docker, DotENV, Error Lens
- [] **MAC-0-A10** — VS Code settings.json configured with formatOnSave, Prettier default formatter, tabSize 2, rulers at 100, TypeScript relative imports, ESLint validation

GitHub Repository Setup (MAC-0-B)

- [] **MAC-0-B1** — Repository cloned via SSH to `~/Projects/HavenWorld`; opens in VS Code
- [] **MAC-0-B2** — Full monorepo directory tree scaffolded (apps/server, apps/client, packages/shared, .github/workflows, .github/ISSUE_TEMPLATE)
- [] **MAC-0-B3** — Root `package.json` created with pnpm workspace config; `pnpm install` runs successfully
- [] **MAC-0-B4** — `.gitignore` includes node_modules, dist, .env, .DS_Store, coverage, .pnpm-store
- [] **MAC-0-B5** — `README.md` created with title, tagline, tech stack, local setup instructions, license
- [] **MAC-0-B6** — `ROADMAP.md` created with Phase 0, Phase 1, and Phase 2 sections using GitHub checkbox format
- [] **MAC-0-B7** — `CONTRIBUTING.md` created with getting started, branching convention, PR requirements, code style
- [] **MAC-0-B8** — `SECURITY.md` created with vulnerability reporting email and supported versions note

- [] **MAC-0-B9** — Bug report, feature request, and PR templates created in `.github/`
- [] **MAC-0-B10** — Branch protection enabled on `main` (PR required, status checks required); `develop` branch created and pushed
- [] **MAC-0-B11** — GitHub Project Board created (HavenWorld Alpha Development); 4 columns; 5 labels created with correct colors

Cloud Service Account Setup (MAC-0-C)

- [] **MAC-0-C1** — Cloudflare account created; Pages project named "havenworld" connected to GitHub repo; build settings configured (`pnpm --filter client build / apps/client/dist`); `NODE_VERSION=22` set; `CF_API_TOKEN` and `CF_ACCOUNT_ID` copied
- [] **MAC-0-C2** — Resend account created; API key (`re_...`) generated and saved; sender domain confirmed as `onboarding@resend.dev`
- [] **MAC-0-C3** — Sentry account created; Organization "HavenWorld" created; `havenworld-server` (Node.js) project created with DSN copied; `havenworld-client` (Browser) project created with DSN copied; auth token for source maps copied
- [] **MAC-0-C4** — UptimeRobot account created; email alert contact added; Discord webhook added as alert contact (add monitors after Part 2 when Oracle VM IP is known)
- [] **MAC-0-C5** — Discord server "HavenWorld" created; all channels in correct categories; all 4 roles created with correct colors and permissions; `@Developer` assigned to yourself; permanent invite links created
- [] **MAC-0-C6** — Hashnode blog created at `havenworld.hashnode.dev`; first post ("Introducing HavenWorld") published; blog URL pinned in Discord `#dev-log`

GitHub Secrets and CI/CD Workflows (MAC-0-D)

- [] **MAC-0-D1** — All 12 GitHub Secrets added: `CF_API_TOKEN`, `CF_ACCOUNT_ID`, `RESEND_API_KEY`, `SENTRY_DSN_SERVER`, `SENTRY_DSN_CLIENT`, `SENTRY_AUTH_TOKEN`, `ORACLE_HOST` (placeholder until Part 2), `ORACLE_SSH_KEY`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `DATABASE_URL`, `REDIS_URL`

- [] **MAC-0-D2** — `deploy-client.yml` created with correct paths filter, pnpm setup, build command, and Cloudflare Pages action
- [] **MAC-0-D3** — `deploy-server.yml` created with test job (lint + test) and deploy job (SSH to Oracle VM, git pull, migrate, build, PM2 restart)
- [] **MAC-0-D4** — `pr-check.yml` created; runs lint, tests, and builds for both server and client on all PRs
- [] **MAC-0-D5** — `docker-compose.dev.yml` created with `postgres:16-alpine` and `redis:7-alpine`; `docker compose -f docker-compose.dev.yml up -d` starts successfully

Landing Page and Legal (MAC-0-E)

- [] **MAC-0-E1** — Landing page (`apps/client/index.html`) created with title, tagline, description, Discord button, GitHub button, Alpha Coming Soon badge, footer with links to Terms and Privacy; dark navy/teal aesthetic
- [] **MAC-0-E2** — Terms of Service (`apps/client/terms.html`) created with Alpha Software Notice, Age Requirement (13+), Acceptable Use, Account Actions, Data Collection, No Warranty, and contact email
- [] **MAC-0-E3** — Privacy Policy (`apps/client/privacy.html`) created with What Is Collected, How It Is Used, Third-Party Services (Sentry, Resend), Data Retention, Your Rights, and No Sale of Data
- [] **MAC-0-E4** — Phase 0 foundation committed and pushed to `develop`; PR created from `develop` → `main`; PR merged; Cloudflare Pages deploy succeeded; landing page live at `havenworld.pages.dev`

Final Verification

- [] **havenworld.pages.dev** loads the landing page in a browser (not a Cloudflare error page)
- [] **GitHub Actions** tab shows at least one completed workflow run (green checkmark)
- [] **Discord server** is accessible; all channels are visible with correct role restrictions

- [] **Hashnode blog** first post is published and publicly accessible
 - [] **All 12 GitHub Secrets** are present in Settings → Secrets and Variables → Actions (ORACLE_HOST can be placeholder until Part 2)
 - [] **No secrets are committed** to the repository — run `git log --all -p | grep -i "re_|password|secret"` and confirm no results
-

Continuation Note — Parts 2, 3, and 4

This is **Part 1 of 4** of the HavenWorld Complete Build Guide. The series continues:

Part

Title

Tags

What It Covers

Part 2

Phase 0: Oracle Cloud VM Setup

LINUX

Oracle Cloud VM provisioning, Ubuntu hardening, PostgreSQL 16 install and database creation, Redis 7 install with password auth, Nginx install and config, Let's Encrypt SSL via Certbot on nip.io domain, PM2 install and ecosystem config, first production deploy, UptimeRobot monitors added, smoke test of full stack

Part 3

Phase 1: Mac Tasks (Client)

MAC

Phaser 3 + Vite 6 client scaffold, all 5 game scenes (Boot, Login, Lobby, Room, UI), 9-layer avatar sprite system with LibreSprite, atlas packing with TexturePacker, 4 room tilemaps with Tiled, Socket.io client connection, avatar customizer UI overlay, friend list overlay, chat UI

Part 4

Phase 1: Linux Tasks (Server)

LINUX

Express + Socket.io server scaffold, Prisma schema (User, Session, Room, Avatar, Furniture, FriendRequest, ChatMessage), invite code registration flow, JWT access + refresh token auth, bcrypt password hashing, RoomManager with player state sync, real-time movement broadcast, chat with profanity filter and rate limiting, personal room with furniture placement, friend system with online status via Redis, Resend email verification integration, Sentry error capture, full production deployment and alpha readiness checklist

Before starting Part 2: Confirm all items on the Phase 0 Mac Completion Checklist (Section 3.6) are checked. Part 2 begins with provisioning the Oracle Cloud ARM VM and requires your GitHub deploy public key (`~/ .ssh/havenworld_deploy.pub`) generated in MAC-0-A6.

HavenWorld Complete Build Guide — Part 1 of 4: Project Foundation & Phase 0 Mac Setup

Developer: Daniel Boone | github.com/dboone323/HavenWorld | havenworld.pages.dev

Document Date: September 18, 2026 | Status: Active — Phase 0 In Progress

Internal development reference. All services referenced are free as of the document date. Free tier limits and availability are subject to change by third-party providers. Verify current limits at each provider's official pricing page before relying on them for production use.